

AWS Security , Monitoring, Scaling, and Billing

Prof. Arnab Mandal

Assistant Professor

CSE Department

Content

1. AWS IAM	3
2. Implements of IAM policies	4
3. Creating and attaching custom policy	10
4 Monitoring Resources	14
5.AWS CloudWatch	14
6 Scaling and Load balancing	15
8.AWS Autoscaling.....	16
9.Cloud Economics and Billing	17
12.Scale and Load Balance your architecture	18

AWS IAM :

In the AWS ecosystem, **Identity and Access Management (IAM)** is the most critical service to master because it is the "gatekeeper" for every other service. If you don't have the right permissions in IAM, you cannot create an S3 bucket, launch an EC2 instance, or access an RDS database.

1. The Four Pillars of IAM

Users

An IAM User represents a **person** or a **service** that interacts with AWS.

Root User: Created when the account is first opened. It has "God-mode" access.

Best Practice: Stop using the Root user immediately after creating your first IAM user.

IAM User: A permanent identity with a name and credentials (password for the Console or Access Keys for the CLI).

Groups

A collection of IAM users.

The Benefit: Instead of giving 50 individual users "Administrator" access one by one, you create an "Admins" group, attach the permission to the group, and add the users to it.

A user can belong to multiple groups (e.g., "Developers" and "Beta Testers").

Roles

Think of a Role as a **"Hat"** that an identity can wear temporarily.

- Roles do not have permanent passwords or access keys.
- **Use Case:** You give an **EC2 instance** a Role so it can "talk" to an S3 bucket without you having to hard-code a password inside the server.
- Roles are also used for **Federation** (letting people log in using their Google or Corporate credentials).

Policies

These are JSON documents that define what is allowed or denied.

- **Structure:** They consist of **Effect** (Allow/Deny), **Action** (What can they do?), and **Resource** (What object are they doing it to?).
- **Managed Policies:** Created by AWS (e.g., Administrator Access).
- **Inline Policies:** Custom-made by you for a specific user or group.

2. Anatomy of an IAM Policy (JSON)

Students often find JSON intimidating, so breaking down the "Effect-Action-Resource" structure is helpful:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::student-example-bucket"
    }
  ]
}
```

3. IAM Authentication vs. Authorization

This is a common exam distinction:

Authentication: "Who are you?" (Username, Password, MFA).

Authorization: "What are you allowed to do?" (Policies and Permissions).

4. Classroom Analogy: The Corporate Office

The IAM User: The employee's ID badge.

The IAM Group: The department (e.g., Marketing, HR).

The IAM Policy: The permissions programmed into the ID badge (e.g., "Can open the front door but cannot enter the server room").

The IAM Role: A "Temporary Visitor Pass" given to a contractor. It works for 8 hours and then expires.

Implement IAM Policies:

Understanding Policy Types

AWS Managed Policies: Created and administered by AWS (e.g., AmazonS3FullAccess). They are easy to use but often too broad.

Customer Managed Policies: Created by the user. These allow for **Fine-Grained Access Control**, which is the gold standard for security.

Lab: Creating and Attaching a Custom Policy

Step 1: Create the IAM Policy (JSON)

1. Navigate to the **IAM Console** and click **Policies** in the left sidebar.
2. Click **Create policy**.
3. Switch to the **JSON** tab and paste the following:

```
{  
    "Version": "2012-10-17",  
    "Statement": [  
        {  
            "Sid": "AllowListBucket",  
            "Effect": "Allow",  
            "Action": "s3:ListBucket",  
            "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME"  
        }  
    ],  
}
```

```
{  
  "Sid": "AllowGetObjects",  
  "Effect": "Allow",  
  "Action": "s3:GetObject",  
  "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"  
}  
]  
}
```

4. Click **Next**, name the policy StudentRestrictedS3Access, and click **Create policy**.

Step 2: Attach the Policy to a User or Group

1. Go to **Users** in the IAM sidebar.
2. Select a specific user (or create a new one called test-user).
3. Under the **Permissions** tab, click **Add permissions** -> **Add permissions**.
4. Select **Attach policies directly**.
5. Search for your custom policy StudentRestrictedS3Access, check the box, and click **Next** then **Add permissions**.

3. Testing the Policy

Now , log in as the test-user and perform these two tests:

The Success Test: Try to open the specific S3 bucket named in the policy. They should be able to see and download the files.

The Failure Test: Try to open **any other** S3 bucket or look at EC2 instances. They should receive an "**Access Denied**" error message.

Monitoring resources :

1. AWS CloudWatch:

CloudWatch is a monitoring and observability service. It provides data and actionable insights to monitor applications, respond to system-wide performance changes, and optimize resource utilization.

Metrics: CPU usage, Disk I/O, Network in/out.

Alarms: Watches a metric over time and performs actions (e.g., "If CPU > 80% for 5 minutes, send an email or trigger Auto Scaling").

Logs: Collects, monitors, and stores log files from your EC2 instances and other services.

Dashboards: Visual representation of your resource health.

2. Scaling and Load Balancing:

Elastic Load Balancing (ELB)

- **Application Load Balancer (ALB):** Works at the Application Layer (Layer 7). It makes smart decisions based on URL paths (e.g., sending /images to one server and /videos to another).
- **Network Load Balancer (NLB):** Works at the Transport Layer (Layer 4). Used for ultra-high performance and static IP addresses.

AWS Auto Scaling

Vertical Scaling: Increasing the "power" of a single server (adding more RAM/CPU).

Horizontal Scaling: Adding **more** servers to the fleet. This is the AWS preferred method.

Desired Capacity: How many servers you want running normally.

3. Cloud Economics and Billing

This is part of the **Cost Optimization** pillar of the Well-Architected Framework. Students need to understand that in the cloud, **Efficiency = Money**.

Key Pricing Models

On-Demand: Pay by the second or hour. No commitment. (Like a taxi).

Savings Plans / Reserved Instances: Commit to a 1 or 3-year term for up to 72% discount. (Like a monthly bus pass).

Spot Instances: Use spare AWS capacity at up to 90% discount, but AWS can "reclaim" them with a 2-minute notice. (Like a "standby" flight ticket).

Billing Tools

AWS Budgets: Set custom budgets that alert you when your costs or usage exceed (or are forecasted to exceed) your budgeted amount.

AWS Cost Explorer: A tool to visualize and manage your AWS spending over time.

AWS Trusted Advisor: An automated "consultant" that scans your account and tells you where you are wasting money or have security holes.

Lab: Scale and Load Balance Your Architecture

We will build a system that can survive a server failure.

The Workflow:

Create an AMI (Amazon Machine Image): Take a "gold image" of a pre-configured web server.

Create a Launch Template: Tell AWS: "Whenever you need a new server, use this AMI, this instance type (t3.micro), and this Security Group."

Create an Auto Scaling Group (ASG): Define a minimum of 2 servers and a maximum of 5.

Attach an ALB: Create a Load Balancer that sits in front of the ASG.

The Test: Have students manually "Terminate" one of their EC2 instances.

The Observation: They will see the Load Balancer stop sending traffic to the dead server, and the Auto Scaling Group will automatically launch a new "replacement" server within minutes.

Key Takeaways:

- IAM: Manage access using the Principle of Least Privilege, using Roles for temporary permissions and Groups for organized user management.
- IAM Policies: Use JSON documents to define "Allow" or "Deny" actions; an Explicit Deny always overrides any "Allow."
- CloudWatch: Monitor resource health via Metrics, collect Logs for debugging, and automate responses using Alarms.
- Elastic Load Balancing (ELB): Distributes incoming traffic across multiple instances to ensure high availability and fault tolerance.
- Auto Scaling: Automatically adjusts the number of EC2 instances to match demand, optimizing both performance and cost.
- Cloud Economics: Switch from upfront hardware costs (CapEx) to a Pay-as-you-go model (OpEx) for compute, storage, and data transfer.
- Billing Tools: Monitor and control spending using AWS Budgets for alerts and Cost Explorer for data visualization.
- Shared Responsibility: AWS secures the physical infrastructure (of the cloud), while you secure your data, code, and configurations (in the cloud).

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

